Framecut / Promo Generator

Project explanation and presentation guide

Purpose: a step-by-step explanation of how this application turns a source video and an editing brief into a short promo.

Based on the repository implementation
Prepared 31 October 2026

1. The short explanation to open with

"Framecut is a web application that creates a short promotional video from a longer video. The user supplies a video or a supported YouTube link, a theme or natural-language editing request, and optional creative, format, and soundtrack settings. The backend analyzes speech, audio energy, motion, and shot changes. It ranks usable moments, plans an edit, and uses FFmpeg to cut and export the result. It can run with local models, or use optional API analysis for a second opinion."

The project is an automated media-analysis and editing pipeline. Its machine-learning models are used for inference. The repository does not train those models.

2. Main backend architecture: models, API, logic, scripts

The backend is organized into four areas with different responsibilities. The important architectural distinction is that the API handles requests, logic performs the work, model files supply pretrained inference assets, and scripts support one-off validation or maintenance tasks.

```
backend/
+-- main.py                Application entry point; mounts UI and API
+-- api/
+-- router.py          HTTP endpoints, uploads, jobs, downloads
+-- logic/
+-- logic.py           Analysis, selection, edit planning, rendering
+-- models/                Cached pretrained model assets (runtime data)
+-- scripts/
+-- validate_synthetic_dialogue.py  Standalone validation utility
+-- storage/               Uploaded media and generated promo files
```

models/ - pretrained model assets

This is a data/cache directory, not a Python package containing model-training code. In this checkout, backend/models/e5-small-v2/ holds the E5 tokenizer/configuration and ONNX model files used for semantic text embeddings. logic.py downloads the configured E5 snapshot on first use if the expected ONNX file is missing, then loads it with ONNX Runtime. Faster-Whisper is initialized from its configured model name when transcription is first needed; its checkpoint is managed by the Faster-Whisper/CTranslate2 model-loading path and may use its own cache. The project does not retrain either model. In Docker, compose.yaml mounts a persistent model volume so downloaded assets can be reused across container restarts.

api/ - the HTTP and job boundary

backend/api/router.py is the interface between the browser and the backend pipeline. It receives and validates uploads, stores source/music files under generated IDs, handles supported YouTube downloads, validates generation settings, creates background jobs, reports job status, and serves generated MP4 files through a constrained download route. It calls logic.generate_promo(...) in a worker thread and sends progress updates into the job record. It does not implement the ranking or

video-editing algorithms.

logic/ - the application pipeline

backend/logic/logic.py contains the core work. It loads the speech and embedding models; probes and extracts audio; transcribes speech; measures sound energy, motion, and shot changes; builds dialogue and visual candidates; scores and selects moments; plans and critiques a draft; handles soundtrack/dialogue timing; renders through FFmpeg; and validates the output. generate_promo(...) is the orchestration function called by the API. The smaller functions in this module implement individual analysis and editing stages.

scripts/ - manual support utilities

backend/scripts/validate_synthetic_dialogue.py is a standalone validation tool, not a server route and not part of every generation request. It creates synthetic speech/video fixtures on macOS, supplies deterministic fixture transcripts, then calls the real generation pipeline across audio modes. It requires FFmpeg, ffprobe, and macOS say. It is useful for exercising end-to-end dialogue/audio behavior without depending on speech-recognition accuracy.

How the folders work together

1. main.py starts FastAPI and registers api/router.py under /api.
2. The frontend calls API routes to upload source media and submit a generation request.
3. The API validates and queues the request, then calls logic.generate_promo(...) in the background.
4. The logic loads cached assets from models/ when needed, analyzes source media, plans and renders a promo, and writes output into storage/.
5. The API reports progress/result metadata and provides the safe download URL; the frontend displays the completed cut.
6. The standalone script can separately exercise the logic pipeline for validation; it is not involved in ordinary browser requests.

In a presentation, describe it as request layer -> orchestration/processing layer -> pretrained model assets and media tools -> stored output -> API response. The model directory is an input to the logic layer, not a separate service that controls the application.

3. Step-by-step: what happens from input to export

Step 1 - The user sets the brief

In the browser, the user chooses a video file or enters a YouTube URL, writes a theme/edit prompt, and selects settings such as target duration, creative direction, aspect ratio, selection strategy, analysis mode, and audio mode. An optional soundtrack can also be uploaded. The frontend checks basic requirements such as file size, presence of a source and a theme, and whether music is required by the selected audio mode.

Step 2 - The backend receives and validates media

backend/api/router.py exposes upload endpoints. Video and music are streamed into per-upload storage with configured size caps. The YouTube endpoint validates/normalizes supported single-video links and downloads the source through yt-dlp. The application probes media metadata and rejects invalid or unsupported inputs. Random IDs and safe path resolution keep uploads and generated jobs isolated.

Step 3 - A background generation job is queued

The frontend sends the upload IDs and selected settings to /api/generate. The API validates the accepted values, creates a job record, and submits work to a thread pool. The browser polls /api/jobs/{job_id} to show progress. The default worker count is one, which limits simultaneous heavy CPU jobs.

Step 4 - The edit request is interpreted

interpret_edit_request turns the natural-language prompt and form settings into bounded, validated controls: theme, length, pacing, content preferences, and narrative direction. When API-assisted analysis is enabled and configured, an optional model may refine those controls. Local rules still provide the offline path, and source timestamps are determined later from media evidence.

Step 5 - Media metadata and audio are prepared

FFprobe/FFmpeg inspect the source. If it has audio, the backend extracts a working audio file and measures short-window RMS energy. If a soundtrack was uploaded, it is inspected separately; its energy, beat accents, sections, and possible tempo are estimated to help choose an excerpt and match moments to music.

Step 6 - Speech is transcribed

In local mode, Faster-Whisper produces timestamped speech segments. In API or hybrid mode, the configured transcription API may be tried first; hybrid mode falls back to local transcription if the API path fails. Segments are merged into dialogue beats and split into sentence-level intervals so selection and final audio handling can preserve complete thoughts. Videos without speech can continue through visual candidates.

Step 7 - Visual rhythm is measured

The local pipeline detects shot boundaries and calculates a motion profile. It scores dialogue intervals for energy, motion, and visual change. It also builds visual-only candidates for useful non-dialogue moments and can keep an opening-shot candidate. This lets the system work when dialogue is sparse or absent.

Step 8 - Optional storyboard analysis adds suggestions

In API/hybrid mode with a configured key, the application samples a bounded storyboard and sends selected frames and context to a vision-capable model. The model can suggest descriptions and moments. These suggestions are mapped onto the source timeline and combined with local evidence; they do not bypass source-duration checks or the local render validations.

Step 9 - Candidates are scored and selected

Dialogue and visual moments receive scores based on the chosen strategy. For dialogue, signals include semantic theme similarity, literal keyword overlap, transcription confidence, language quality/coherence, energy, visual change, motion, optional API relevance, and soundtrack fit. The selector also considers duration, coverage, diversity, and narrative roles. If no dialogue is found, the theme strategy is switched to visual-montage behavior.

Step 10 - A structured draft is assembled and critiqued

The system expands selected timestamps with safe context, snaps cuts toward shot boundaries, repairs dialogue edges, separates overlapping context, and fits the clip set to the time budget. It checks whether the draft has adequate coverage and whether speech was cut or a transcription artifact was selected. It can repair the draft for a bounded number of passes (default three; maximum five).

Step 11 - The soundtrack and dialogue timeline are prepared

For music modes, the system selects a suitable excerpt and maps dialogue intervals onto the output timeline. It identifies source-audio intervals that should be muted to avoid leaving an unintended partial spoken line across a cut. It validates dialogue timing against the selected clips and audio mode before rendering.

Step 12 - FFmpeg renders the promo

The source clips are stitched into an MP4. Output framing can preserve source dimensions, crop to vertical 9:16, or crop to square 1:1. Audio can use the source, external music, a mix, or dialogue-focused mixing. Music can be ducked under selected dialogue where the chosen mode calls for it.

Step 13 - Render quality checks run

The application checks that the output can be probed/decoded, that its duration is within the expected range, that the aspect ratio is correct, and that structural/dialogue timeline validations passed. This is media validation; it does not perform speech recognition on the final rendered video or replace a human listening review.

Step 14 - The result is returned

The job status becomes complete and the API returns a safe download URL plus metadata: selected clips, source and output timestamps, script outline, analysis provider, iteration trace, soundtrack details, and QA report. The frontend displays the video, cut sheet, script outline, and generation summary.

4. Models and what "training" means here

Faster-Whisper (default model size: base)

This is the local speech-recognition component. It is a faster implementation of Whisper using CTranslate2. The application loads the selected pretrained checkpoint and runs transcription. It does not fine-tune Whisper on project videos. The default device is CPU and the default compute type is int8, a practical memory/speed choice for CPU inference. A different model size, device, or compute type can be set through environment variables.

E5 sentence embeddings (default: Xenova/e5-small-v2)

This pretrained embedding model represents theme queries and dialogue passages as vectors. The code uses the model’s recommended query: prefix for the theme and passage: prefix for dialogue, tokenizes up to 256 tokens, mean-pools token outputs using the attention mask, and normalizes vectors. Cosine similarity is then used as a semantic relevance signal. The ONNX model runs through ONNX Runtime on CPU. On first use, model files are downloaded and cached under the model directory. If loading/inference fails, literal keyword overlap is the fallback. The embedding model is not trained here.

Optional API models

When configured, the API transcription model (whisper-1 by default) can provide speech transcription, while the vision-capable model (gpt-4.1-mini by default) can reason over sampled storyboard frames and the editing request. They are optional services; the local pathway remains available. API requests can send extracted audio chunks and storyboard images, rather than uploading the original video as a single opaque asset.

Are hyperparameters trained or tuned?

There is no dataset-driven hyperparameter training or automatic optimization loop in this repository. The "hyperparameters" visible here are inference settings and manually chosen decision parameters, not learned weights. Examples include model size, CPU/int8 inference, maximum token length, ranking weights, clip lengths, audio-analysis window size, repair-pass limit, and worker count. The score weights in score_dialogues are explicit constants selected in code for each strategy. The draft critic can repair an individual edit, but it is not model training or parameter tuning.

5. How the ranking works, in plain language

For each dialogue beat, the code calculates normalized indicators: semantic match, word overlap, ASR confidence, sentence quality/coherence, audio energy, visual change, motion, API score, and (when available) soundtrack match. A strategy selects a fixed weighting profile:

Strategy * What it favors
theme * Strong theme relevance, with supporting quality and audiovisual signals
best-overall * Strong general moments, with less dependence on theme wording
story-arc * Theme relevance plus narrative and story coverage
visual-montage * Visual change, motion, and energy; useful for montage or no-dialogue input

The resulting score is one part of selection. The selector also tries to cover the target duration, avoid redundant moments, preserve narrative order when requested, and assign beats roles such as hook, setup, escalation, or ending. These choices are rule-based heuristics, not a separately trained ranking model.

6. User-facing configurations

Analysis mode

* Local: uses local transcription and local visual/audio analysis. This is the offline-first option after model files are cached.
* Hybrid / auto: uses API-assisted transcription/storyboard analysis when a key is configured, then falls back to local analysis for supported failures.
* Vision + audio API: explicitly requires a configured API key; API errors are surfaced rather than silently treated as a successful API run.

Selection strategy

* Theme + story arc (theme): balances the brief with story coverage; default.
* Best overall (best-overall): favors broadly strong audiovisual/dialogue moments.
* Story arc first (story-arc): emphasizes a coherent progression.
* Dialogue + visual peaks (visual-montage): gives greater priority to motion and visual change.

Creative direction

* Balanced: moderate context and clip-length limits.
* Cinematic: fewer, longer moments and more surrounding context.
* High impact: shorter clips and tighter pacing.

The implementation uses different context padding and maximum clip durations for these directions; the target duration remains one of 15, 30, 45, 60, 90, or 120 seconds.

Aspect ratio

* Source: preserve the source framing.
* Vertical: crop/reframe for 9:16.
* Square: crop/reframe for 1:1.

Audio mode

* Auto: use source audio by default; use a mixed soundtrack when music is supplied.
* Source: original source audio only; an uploaded music track is not used.
* Music: external music only; requires a soundtrack.
* Mixed: soundtrack with source audio; requires a soundtrack.
* Dialogue: source audio/dialogue is foregrounded with lower music when supplied.

The backend validates incompatible combinations. Selected dialogue is mapped to output time so soundtrack ducking and safe source-audio muting can be applied.

7. Important files and their responsibilities

File * Responsibility
frontend/index.html * Web form, controls, progress display, preview, download, cut sheet, script and agent report sections.
frontend/js/index.js * Validates browser inputs, uploads media, submits generation settings, polls job status, and renders the result.
frontend/css/index.css * Layout, styling, and responsive behavior of the web interface.
backend/main.py * Creates the FastAPI app, mounts API routes and frontend assets, and serves the home page.
backend/api/router.py * Upload and URL-download endpoints, validation, isolated storage paths, job queue/status, and safe promo downloads.
backend/logic/logic.py * Main media pipeline: models, transcription, analysis, scoring, edit planning, FFmpeg rendering, and output validation.
backend/tests/test_logic.py * Unit tests for logic and safety behavior.
backend/scripts/validate_synthetic_dialogue.py * macOS validation utility that creates synthetic speech/media fixtures and exercises audio modes through the generation pipeline; requires FFmpeg, ffprobe, and say.
backend/requirements.txt * Pinned Python dependency list for a pip-based setup.
backend/pyproject.toml * Project metadata and dependency declarations for uv.
backend/uv.lock * Locked dependency resolution used by the Docker build for reproducibility.
backend/Dockerfile * Builds the backend image, installs FFmpeg and Node.js, syncs Python dependencies, and launches Uvicorn.
compose.yaml * Runs the container, exposes local port 8000, persists storage/model cache volumes, and configures health checks/API settings.
README.md * Setup instructions, product capabilities, environment variables, modes, and usage notes.
example_video/avengers.mp4 * Example source video for trying the application.
generated_example_promo/promo.mp4 * Example rendered output.

8. Runtime configuration reference

Variable * Default * Effect
PROMO_WHISPER_MODEL * base * Local Faster-Whisper model size/checkpoint.
PROMO_WHISPER_DEVICE * cpu * Inference device for Faster-Whisper.
PROMO_WHISPER_COMPUTE_TYPE * int8 * CTranslate2 compute/quantization mode.
PROMO_EMBEDDING_MODEL * Xenova/e5-small-v2 * Hugging Face embedding model ID.
PROMO_MODEL_DIR * backend/models * Local cache directory for embedding model files.
PROMO_STORAGE_DIR * backend/storage * Upload and generated output storage root.
PROMO_WORKERS * 1 * Maximum background generation threads; clamped to at least one.
PROMO_MAX_UPLOAD_BYTES * 1073741824 (1 GiB) * Maximum video upload size.
PROMO_MAX_MUSIC_UPLOAD_BYTES * 536870912 (512 MiB) * Maximum soundtrack upload size.
PROMO_VISION_API_KEY * unset * Enables optional API-assisted analysis. OPENAI_API_KEY is also accepted by the logic module.
PROMO_VISION_MODEL * gpt-4.1-mini * Vision-capable model for storyboard reasoning.
PROMO_API_TRANSCRIBE_MODEL * whisper-1 * API speech transcription model.
PROMO_API_TIMEOUT_SECONDS * 120 * Request timeout for API calls.
PROMO_YOUTUBE_MAX_DURATION * 7200 seconds * Maximum accepted YouTube source duration.
PROMO_MAX_REPAIR_ITERATIONS * 3 * Draft critic passes; clamped to the range 1-5.

In Docker Compose, the documented API and music-upload values are passed through the environment. Other variables can be added to the service environment as needed.

9. Suggested presentation script

1. Introduce the goal: "This system automates the first cut of a promo from a longer video, guided by a theme and optional editing preferences."
2. Show the user inputs: Explain source, theme, duration, direction, aspect ratio, strategy, analysis mode, and soundtrack.
3. Explain the model choice: Faster-Whisper detects speech; E5 represents the theme and dialogue for semantic matching. Both are pretrained inference models.
4. Explain the analysis signals: Speech is combined with confidence, sentence quality, audio energy, shot changes, and motion. Optional API storyboard analysis adds visual descriptions.
5. Explain selection: Fixed strategy weights rank moments; coverage, diversity, and narrative structure help make a coherent sequence.
6. Explain assembly: The system fits clips to the time budget, protects dialogue boundaries, aligns optional music, and renders with FFmpeg.
7. Explain validation and output: It checks duration, aspect ratio, decoding, and timeline constraints, then returns the promo and a cut sheet.
8. Clarify training: "The repository does not train neural networks. We use pretrained models and configure inference/runtime settings and hand-designed ranking weights."

10. Practical limits to mention

* This is an automated first edit; creative quality still depends on the source material and specificity of the prompt.
* Semantic similarity is a relevance clue, not proof that the system understands a film’s full plot.
* Local operation may require downloading model files the first time; afterward cached models can be used without API analysis.
* API modes depend on network access, valid credentials, and external service availability.
* Render validation checks media and timeline properties, but it does not replace watching and listening to the final promo.

11. One-sentence summary

Framecut combines pretrained speech and text models with deterministic audio/visual measurements and rule-based editing heuristics, then uses FFmpeg to render and validate a theme-guided promo.